

ДЗ 2. Hadoop экосистема и MapReduce

Формулировка задания

Цель: освоить модель MapReduce и её ограничения.

Задание: реализовать задачу подсчёта TF-IDF или топ-N слов по корпусу текстов с использованием Hadoop MapReduce. Объяснить, как происходит shuffle и sort. Сравнить с локальной реализацией.

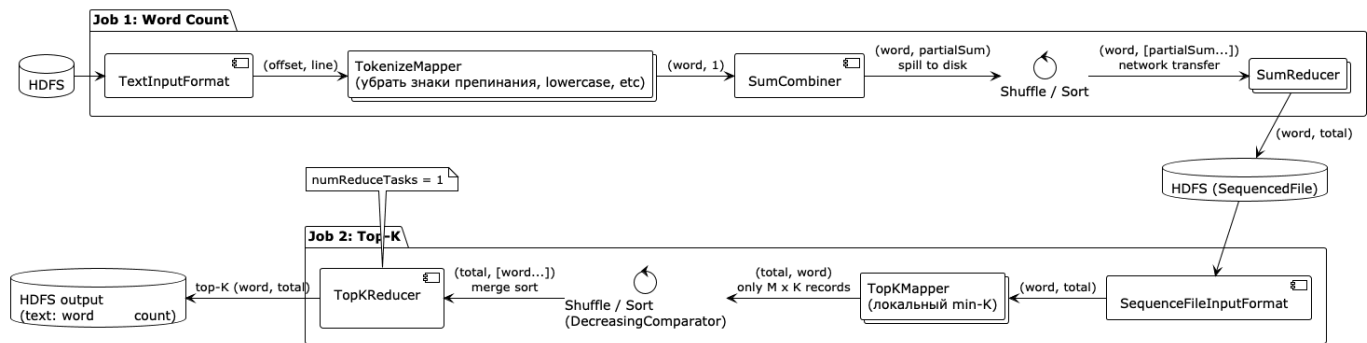
Форма отчетности: в отчёте должно содержаться описание используемых данных и решаемой задачи, а также реализованного MapReduce-решения. Необходимо описать работу Mapper и Reducer, привести промежуточные пары (key, value) и объяснить роль этапа Shuffle/Sort в решении задачи. Также необходимо описать запуск MapReduce-задачи, привести пример полученного результата и сравнить его с результатом и временем выполнения локальной реализации.

Критерии (max 6 б.): корректная реализация алгоритма TF-IDF или топ-N слов на Hadoop MapReduce, верные результаты работы кода, подробное объяснение этапов shuffle и sort, обоснованное сравнение с локальной реализацией, отчет содержит корректное описание механизмов обеспечения надежности и масштабируемости HDFS.

Выбранная задача: Топ-K слов по корпусу текстов

Архитектура решения

1. Загружаем набор текстов с HDFS, используем `TextInputFormat`. В mapper попадает (offset, line).
2. Токенизируем текст в `TokenizerMapper` - слова приводятся к нижнему регистру, цифры знаки препинания и прочее выкидываются. Маппер не выполняет подсчетов и выдает (word, 1).
3. Считает локальную сумму по словам в `SumCombiner` (`== SumReducer`). На выход (word, partialSum).
4. Shuffle and sort. Сортировка на этом этапе крайне важна, тк она гарантирует, что все вхождения ключа попадут на 1 reducer, без этого 2 map-reduce job'a не будет работать корректно.
5. Считаем глобальную сумму по словам, на данном этапе у нас набор бинарных файлов, содержащий KV `<word, sum>`
6. Сбрасываем на диск и запускаем вторую джобу
7. Вычитываем данные в формате для KV `SequenceFileInputFormat` - специальный формат бинарных данных hadoop.
8. Маппер в памяти держит top-k при помощи heap, при завершении сбрасывает кучу в output. Исходим из того, что K не огромно и помещается в памяти одного маппера. На выходе получаем $\leq K$ наиболее часто встречающихся слов попавших на этот маппер. Из-за сортировки в прошлой джобе слово попадает максимум в 1 маппер. Используется композитный ключ `CountWordKey` для того, чтобы делегировать глобальную сортировку hadoop.
9. Shuffle and sort. Сортирует все heap'ы, на выходе нам остается только взять top-k из них.
10. `TopKReducer` Берет первые K значений, тк сортировка уже готова. Число reducer'ов = 1, для того, чтобы получить финальный результат.



Инфраструктура

Hadoop в докере с прошлой лабы, подробнее - [compose.yaml](#) и [hadoop.env](#)

Используемые данные

Для анализа используется 10 книг на английском языке, размер файлов небольшой, но нам хватит.

Названия и размер книг [books.csv](#).

```

> chmod +x download_books.sh
  ./download_books.sh

```

Загружаем данные в hdfs

```

> docker exec -it namenode sh
  hdfs dfs -put /opt/input /input/books
  hdfs dfs -ls /input/books

```

Токенизация

Токенизация реализована в `Tokenizer.java`. Словом считается любая комбинация удовлетворяющая regex'y `"[A-Za-z]+(?:'[A-Za-z]+)*"`. Опционально можно включить модель `stanfordNLP` для дедупликации слов, например в базовом варианте `whale` и `whales` это два разных слова, при использовании модели `whales` будет нормализовано до `whale`.

Дополнительно производится ряд нормализацией, например слова с "ударением" нормализуются, подробнее в `TokenizerTest.java`.

Запуск Job'ы

```
► cd jobs
./gradlew jar
docker exec -it resourcemanager yarn jar /opt/jobs/topk-jobs.jar /input/books /output/topk 10 2
```

Формат вызова:

```
yarn jar topk-jobs.jar [-D prop=value] input output [k] [reducers] [useCombiner] [cleanTmp]
```

Параметр	По умолчанию	Описание
input	—	Директория с текстами в HDFS, например /input/books
output	—	Директория для результата в HDFS. Не должна существовать.
k	10	Сколько самых частых слов вывести
reducers	1	Число reducer'ов в job 1 (word count). В job 2 reducer всегда один, иначе не получить глобальный top-K
useCombiner	true	Включить SumReducer как combiner в job 1. С false удобно сравнивать объем shuffle
cleanTmp	true	Удалить промежуточный результат job 1 (/tmp/topk-wordcount-<uuid> , путь печатается в лог). false - оставить для отладки

Через -D передаются свойства Hadoop и настройки job:

Свойство	По умолчанию	Описание
topk.lemmatize	true	Лемматизация через StanfordNLP (whales -> whale). false - только regex

k задается только позиционным аргументом: -D topk.k=... перезаписывается значением k .

Примеры:

```
► docker exec -it resourcemanager yarn jar /opt/jobs/topk-jobs.jar -D topk.lemmatize=false /input/books
/output/topk 20 3 false
docker exec namenode hdfs dfs -rm -r /output/topk
```

Прогресс и счетчики - в YARN UI <http://localhost:8088>.

Результат:

```
► docker exec namenode hdfs dfs -cat /output/topk/part-r-00000
```

Результат

- [пример логов](#)
- [логи с применением combiner'a](#)
- [top100](#) (Combine input records=2604933, Combine output records=121097)
- [top100 + нормализация](#) (Combine input records=2621580, Combine output records=88617)

Всего слов (счетчики job 1: Map output records и Reduce input groups):

Режим	Всего слов	Уникальных слов
без нормализации	2 604 933	46 605
с нормализацией	2 621 580	33 389

С нормализацией слов больше, тк сокращения разбиваются на 2 слова: he's -> he , be , don't -> do , not .

Наглядно влияние нормализации заметно на примере be , в обычной версии вхождение слова разделено на

- was - 22313
- is - 21184
- be - 16336
- are - 8258
- were - 7976
- been - 5024
- am - 4493
- etc

В нормализованной версии только be - 92633

Надежность и масштабируемость

```
dfs.replication=2
```

Глобально тема рассматривалась в [первой лабораторной работе](#), не вижу смысла повторяться.

Сравнение с локальной реализацией

Локальная однопоточная реализация - LocalTopK.java : один процесс без Hadoop, тот же Tokenizer и та же сортировка (count desc, word asc), подсчет в HashMap , top-K через heap размера K.

```
► cd jobs
./gradlew jar
java -cp build/libs/topk-jobs.jar com.quicklybly.bigdata.topk.LocalTopK ../input 100 false
java -cp build/libs/topk-jobs.jar com.quicklybly.bigdata.topk.LocalTopK ../input 100 true
```

Результат

Результаты полностью совпадают: top-100 локальной версии построчно совпадает с `output/top100.txt` и `output/top100-lemmatize.txt` в обоих режимах, совпадает и число слов (2 604 933 / 2 621 580) и уникальных слов (46 605 / 33 389).

Производительность

Локально - один поток на хосте (Apple M5 Pro, 48GB), Hadoop - кластер в docker на том же ноутбуке (18CPU, 8GB allocated to whole docker vm), k=100, combiner включен.

Режим	Локально	Hadoop	Hadoop медленнее
без нормализации	0.6 с	~63 с (job 1 ~38 с, job 2 ~22 с)	~100 раз
с нормализацией	44 с	~115 с (job 1 ~90 с, job 2 ~22 с)	~2.6 раза

Почему Hadoop медленнее

- Объем данных: ~15 МБ текста легко помещается в память одной машины, hadoop создает по мапперу на каждый блок (10 мапперов), в итоге огромные накладные расходы на инициализацию и прогрев приложения
- Дополнительные расходы на "flush" в hdfs, нужно записать файл с репликацией и вычитать его обратно в память.
- Кластер по факту работает на одной и той же машине, к тому же с виртуализацией (образы под amd) и зарезанными лимитами.
- С нормализацией каждая map-задача заново загружает модель StanfordNLP (10 раз).

Вывод

Разрыв сокращается со ~100 раз до ~2.6 раза, когда растёт объем вычислений на одну запись: накладные расходы Hadoop постоянны, а полезная работа распараллеливается. Hadoop выигрывает, когда:

- данные не помещаются на одну машину
- обработка каждой записи тяжелая, а ноды реальные - задачи выполняются параллельно на разных машинах рядом с учетом locality
- нужна отказоустойчивость: упавшая задача перезапускается на другой ноде, данные защищены репликацией HDFS
- нужно горизонтальное масштабирование из коробки

На корпусе из 10 книг правильный выбор - локальная реализация.